Git Logs & History

1 Comment / By Dot Net Tutorials / February 22, 2026

Back to: Microservices using ASP.NET Core Web API Tutorials

Git Logs & History

In real projects, Git history isn't something we read just to know about old commits. We use it when a bug appears, when a controller suddenly stops working, when business logic changes unexpectedly, or when we want to know who modified a file or a line of code. That is the real use of Git logs and history.

What Is Git History?

In practical words, Git history is the record of all commits made in your project. Every time you commit, Git stores that change as one checkpoint. Over time, those checkpoints create the development timeline of your application.

For example, suppose in your ASP.NET Core Web API project, you made these commits:

- Initial Product API setup
- Add CategoriesController
- Add Product validation
- Change ProductService logic
- Fix bug in price validation

Now, if a bug appears after the last two changes, Git history helps you go back and check:

- What changed
- When it changed
- Who changed it
- Which file was modified
- What exact lines were added or removed

That is the real meaning of Git history.

Why Commit History Is Important

Commit history is important because projects keep changing. Code does not stay fixed. New features come in, old logic gets modified, validations change, routes change, services change, and sometimes bugs get introduced by mistake.

When something goes wrong, history helps answer practical questions like:

- Which commit introduced the problem?

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Which file was changed?
- Was it changed yesterday or last week?
- Was it a controller change or a service change?
- Who made the change?
- Can we compare the old and new versions?

Commit history is your investigation tool. Whenever something breaks, Git history becomes one of the first places to check.

Real Project Scenario

Suppose your POST /api/products endpoint was working fine earlier. Today, after pulling the latest changes, it starts accepting invalid prices again. Now, instead of guessing, you use Git history like this:

- Open repository history
- Check recent commits
- Find commits related to ProductsController.cs or ProductService.cs
- Inspect the suspicious commit
- Compare it with the previous version
- Identify the exact line that changed

This is how Git history is used in real work.

How to View Commit History in Visual Studio

In Visual Studio, the most practical place to see project history is the **Git Repository** window.

Step 1: Open the Project

Open your solution in Visual Studio.

Step 2: Open Git Repository Window

From the top menu:

- Click **View**
- Click **Git Repository**

Now Visual Studio will show the current branch and its commit history.

Step 3: Check the Commit List

You will see the commits made in that branch. Each commit usually appears with:

- Commit message

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Author name
- Date/time
- Commit ID or short hash

Step 4: Open a Commit

Click a commit to inspect it.

You can usually see:

- Which files changed
- What lines were added
- What lines were removed

This is the practical way to inspect project history.

What Information a Commit Shows

A commit is not just one message line. In practical use, a commit shows you the full story of one code change.

When you open a commit, you usually see:

- Commit message: Example: Add validation to product POST action
- Author: Who made the commit
- Date and time: When the commit was created
- Commit ID / Hash: Unique identity of that commit
- Changed files: Example:
    - ProductsController.cs
    - ProductService.cs
- Code difference: Which lines were added, modified, or removed

A commit is a saved change in the project, along with its explanation and the code changes.

How Do We View the History of a Specific File?

This is one of the most useful features in real projects. Suppose the issue is clearly inside:

- ProductsController.cs
- ProductService.cs
- Program.cs
- appsettings.json

In that case, you do not need the full repository history first. You need the history of that specific file.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Step-by-Step in Visual Studio

Step 1: Open Solution Explorer

Find the file you want to inspect.

Example:

- Controllers/ProductsController.cs

Step 2: Right-Click the File

Right-click that file.

Step 3: Open File History

Choose:

- **Git**
- **View History**

Now Visual Studio will show only the commits that affected that file. This is extremely useful because now you can focus only on changes made to that controller or service.

Practical Use

Suppose ProductsController.cs started returning wrong responses. Now open the file history and check:

- Which recent commits touched that controller
- What was changed in the post-action
- Whether validation logic was removed or changed

This is much faster than checking the full repository history.

How to Delete a Repository in Git and Also in GitHub

Part 1: Delete a Local Git Repository

Suppose your project exists on your machine and Git is initialized in it. If you want to remove Git tracking while keeping the project files, delete the .git folder from the project root. It means:

- Project files remain
- Git tracking is removed
- The folder is no longer a Git repository

Part 2: Delete a Repository in GitHub

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

If the repository is hosted on GitHub and you want to remove it online, do this:

1. Open the repository on GitHub
2. Click **Settings**
3. Scroll down to **Danger Zone**
4. Click **Delete this repository**
5. Confirm the repository name
6. Complete the deletion

It means:

- The GitHub repository has been deleted
- Remote code is removed
- Collaborators lose access to that remote repo

Conclusion

Git history is one of the most useful tools in real development. It helps us see what changed, who changed it, and when. When a bug appears or something stops working, history makes investigation faster and easier.

So, instead of treating Git logs as theory, we should use them as a practical tool for debugging, comparing changes, checking file history, and understanding project progress. The better we use Git history, the easier it becomes to maintain and troubleshoot real projects.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent | Do Not Sell My Information

Explore our developer-friendly HTML to PDF API | Printed using PDFCrowd | HTML to PDF